

Lo que falta para que funcione todo

Auditado ejecutando `npm run doctor`, comprobando las tablas de Supabase y revisando cada flujo de la web. **Casi todo el trabajo pendiente es pegar claves, no programar.** Dos claves desbloquean la mitad de la lista.

YA FUNCIONA, SIN TOCAR NADA

- ✓ Login y registro con email en los dos portales
- ✓ Panel de empresa: ofertas, pool, desbloqueo, ficha de contacto
- ✓ Ajustes de cuenta: perfil, contraseña, sesiones, privacidad, exportar datos
- ✓ Retos y evaluación local, ranking, progreso en el navegador
- ✓ Las tres tablas de Supabase (`profiles` , `evaluations` , `credentials`) ya existen

BLOQUEADO AHORA MISMO

- × Evaluación por IA de los retos — sin clave de Anthropic
- × Guardar progreso en la nube — sin service key
- × Eliminar cuenta — sin service key
- × Botón de Google — proveedor sin activar
- × Credencial en blockchain — sin wallet ni contrato
- × Emails y cobros reales — sin servicios contratados

Etiquetas:

BLOQUEA

CONFIG

CONSTRUIR

OPCIONAL

FASE 1 Dos claves en el `.env`

10 minutos · es lo que más desbloquea de toda la lista

SUPABASE_SERVICE_KEY

BLOQUEA 4 COSAS

Supabase → *Project Settings* → *API* → copia la **service_role** (la secreta, no la anon) y pégala en el `.env`.

Desbloquea: guardar el perfil en la tabla `profiles`, sincronizar el progreso entre dispositivos, emitir credenciales y eliminar cuentas. Ahora mismo `delete-account` responde con un 503 explícito porque esta clave está vacía.

ANTHROPIC_API_KEY

BLOQUEA EL NÚCLEO DEL TFM

`console.anthropic.com` → *API Keys*. Sin ella, corregir un reto lanza un error: no hay evaluación falsa de reserva, y eso está bien.

Sobre el modelo: el `.env` apunta a `claude-sonnet-4-6`, que sigue siendo un modelo vigente — puedes dejarlo tal cual. La función además prueba modelos alternativos si el configurado falla. Coste orientativo: unos céntimos por cada decena de evaluaciones.

Comprueba las dos de golpe:

```
npm run doctor
```

```
# ahora: 2 bloqueos y 1 aviso — con las dos claves baja a 1 bloqueo (blockchain)
```

FASE 2 Acceso con Google y salto a producción

15 minutos · guía detallada aparte

Activar el proveedor Google

CONFIG

Crear la app en Google Cloud, pegar Client ID y Secret en Supabase y añadir las Redirect URLs. Tienes la guía paso a paso con tus valores exactos.

Verifica con: `npm run check:google` — te dice cuál de los tres pasos falta.

Las mismas variables en Netlify

CONFIG

El `.env` es solo para tu máquina. En *Netlify* → *Site settings* → *Environment variables* hay que repetir `ANTHROPIC_API_KEY`, `SUPABASE_URL` y `SUPABASE_SERVICE_KEY`, o la web publicada seguirá sin guardar nada.

Site URL y Redirect URLs de producción

CONFIG

En Supabase → *Authentication* → *URL Configuration*, con el dominio real de la web. Sin esto, quien entre con Google desde producción acabará en la portada sin sesión.

Revisar las políticas RLS

VERIFICAR

Las tres tablas responden a la clave pública. Están vacías, así que no he podido distinguir “RLS bloquea” de “no hay filas”. Conviene confirmarlo antes de meter datos reales: la premisa del producto es el anonimato.

```
-- En Supabase → SQL Editor
select tablename, rowsecurity from pg_tables where schemaname='public';
select tablename, policyname, cmd from pg_policies where schemaname='public';
```

Qué buscar: `rowsecurity = true` en las tres tablas y al menos una política por tabla. Si sale `false`, cualquiera con la clave anon (que es pública, está en el HTML) podría leer emails y teléfonos.

FASE 3 Servicios que hay que contratar

Aquí sí hay que programar. Nada de esto hace falta para defender un TFM

Envío de emails

CONSTRUIR

No hay proveedor de correo. Las preferencias de notificación se guardan en la cuenta pero no sale ningún email. Haría falta contratar Resend o SendGrid y añadir una función `send-notification` que se llame al evaluar un reto y al desbloquear un contacto.

Ojo: Supabase sí manda sus propios correos (confirmar cuenta, cambiar email). Eso ya funciona; lo que falta son los del producto.

Pasarela de pago

CONSTRUIR

Los €49 del desbloqueo son simulados: quedan registrados en el historial de la empresa pero no generan ningún cargo. Con Stripe serían una función de checkout, un webhook de confirmación y guardar el desbloqueo solo cuando el pago se confirme.

2FA, registro de actividad y planes Pro

OPCIONAL

Los tres salen marcados como *No disponible* en los ajustes. El 2FA es el más asequible: Supabase trae la API TOTP hecha y solo falta la interfaz de QR y verificación.

FASE 4 De demo a producto multiusuario

El cambio conceptual más grande que queda

Las ofertas viven en el navegador

CONSTRUIR

Cuando una empresa publica una oferta se guarda en `localStorage`. Es decir: **nadie más la ve**. Para que un candidato en otro ordenador la encuentre hace falta una tabla `jobs` en Supabase y sus funciones de lectura y escritura.

Lo mismo pasa con los desbloques de contacto: son locales, así que dos empresas distintas no comparten estado.

El pool de candidatos es de demostración

CONSTRUIR

La lista que ve la empresa son perfiles inventados en el código. Los candidatos reales solo aparecen como tu propia ficha. Para un pool de verdad, la consulta debería salir de `profiles` + `evaluations` con las políticas RLS que garanticen el anonimato.

Métricas de negocio del panel admin

OPCIONAL

MRR, ARR y churn son valores fijos en el código. El bloque de IA y costes sí usa datos reales de las evaluaciones registradas. Para un TFM está bien así, pero conviene decirlo en la memoria en lugar de que lo pregunte el tribunal.

FASE 5 La credencial en blockchain

30 minutos · aparcado a propósito para el final

Wallet de testnet y fondos

CONFIG

Va al final por decisión tuya. Nada de las fases anteriores depende de esto, y la web funciona entera sin ello: el botón de sellar la credencial simplemente avisa de que no está configurado.

MetaMask → wallet **nueva**, solo para pruebas → exportar clave privada a `ISSUER_PRIVATE_KEY` y la dirección pública a `ISSUER_ADDRESS`. Luego pedir POL de prueba en el faucet de Polygon.

Nunca uses una wallet con fondos reales: la clave privada acaba en un archivo de texto.

Desplegar el contrato

UN COMANDO

```
npm run compile:contract # prueba en seco, no gasta nada
npm run deploy:contract  # imprime SKILLPASS_CONTRACT_ADDRESS
```

Esa dirección se pega en el `.env` y ya se pueden anclar credenciales.

El RPC no responde

VERIFICAR

`npm run doctor` falla con `ENOTFOUND rpc-amoy.polygon.technology` y desde mi entorno tampoco lo alcanzo. Puede ser una restricción de red mía o que el endpoint público esté caído.

Pruébalo en tu máquina. Si también falla, pon un RPC de Alchemy o Infura en `POLYGON_AMOY_RPC` — el público es conocido por ser inestable.

Por dónde empezar

Pega las **dos claves de la fase 1** y ejecuta `npm run doctor`. Con eso pasas de “demo bonita” a “la IA corrige de verdad y el progreso se guarda en la nube”, que es el 80% del valor demostrable del proyecto.

Con las fases 1 y 2 el recorrido principal queda cerrado: candidato → reto → evaluación por IA → skill verificada → empresa que la consulta. Eso ya es defendible tal cual.

Las fases 3 y 4 son producto real, no TFM. Y la 5, la blockchain, queda para el final: es la guinda del relato — la credencial verificable — pero no bloquea absolutamente nada de lo anterior.